
Versionierung

Kriterium	Git	Mercurial (Hg)	Subversion (SVN)
Geschwindigkeit	Sehr schnell	Schnell, aber langsamer als Git	Langsamer als Git und Hg, besonders bei großen Repos
Handhabung	Steile Lernkurve, aber mächtig	Benutzerfreundlicher als Git	Einfach für Anfänger, aber weniger flexibel
Zusammenarbeit	Dezentral, jeder hat eine Kopie des Repos	Dezentral, jeder hat eine Kopie des Repos	Zentralisiert, alle arbeiten mit dem Hauptserver
Offline-Arbeit	Vollständig möglich	Vollständig möglich	Eingeschränkt
Popularität	Sehr hoch (Standard für viele Open-Source-Projekte)	Weniger verbreitet als Git	Rückläufig, früher weit verbreitet
Hosting-Plattformen	GitHub, GitLab, Bitbucket	Bitbucket (eingestellt für Hg), Phabricator	SVN-Server, Apache Subversion, Cloud-Dienste
Erfahrung	Bereits vorhanden	Einarbeitung notwendig	Einarbeitung notwendig

Ausgewähltes Tool: Git

Begründung:

- Ermöglicht schnelle, flexible und dezentrale Entwicklung
- Ist der De-facto-Standard bei Open-Source-Projekten
- Benötigt keine zusätzliche Einarbeitung der Teammitglieder, da bereits Erfahrungen vorliegen

UML-Tool mit Quellcodegenerierung

Kriterium	Papyrus	StarUML	Visual Studio (VS UML Tools)
Typ	Open-Source UML-Modellierungswerkzeug	Kommerzielle UML-Software mit Open-Core	Integriert in Visual Studio (ältere Versionen) oder durch Erweiterungen
Quellcodegenerierung	Ja (Java, C++, SysML)	Ja (C#, Java, C++, Python über Plugins)	Ja (mit UML-Klassenmodellen für C# und .NET)
Handhabung	Komplex, da stark in Eclipse integriert	Einfach und intuitiv	Sehr einfach für .NET-Entwickler
Integration mit IDEs	Eclipse	Eigenständig, aber kann mit VS Code oder IntelliJ über Plugins genutzt werden	Integriert in Visual Studio (ältere Versionen)

Kriterium	Papyrus	StarUML	Visual Studio (VS UML Tools)
Erweiterbarkeit	Ja, über Eclipse-Plugins	Ja, mit eigenen Plugins und Skripten	Eingeschränkt, nur über VS-Extensions
Lizenzmodell	Open Source	Kommerzielle Lizenz	Teil von Visual Studio
Erfahrung	Bereits vorhanden	Einarbeitung notwendig	Einarbeitung notwendig

Ausgewähltes Tool: Papyrus

Begründung:

- Open Source und direkt in die Eclipse-IDE integrierbar
- Unterstützt Quellcodegenerierung für Java, was Entwicklungsprozess beschleunigt
- Alle Teammitglieder sind mit diesem Tool bereits vertraut

Build-Tool

Kriterium	Maven	Gradle	Ant
Architektur	Deklarativ mit POM (Project Object Model)	Skriptbasiert (Groovy/ Kotlin DSL)	Task-basiert (imperativ)
Abhängigkeitsverwaltung	Automatisch	Automatisch & effizienter als Maven	Manuell
Lernkurve	Mittel (standardisierte Struktur)	Höher als Maven (Skripting erforderlich)	Hoch (aufwendig durch XML-Skripte)
Geschwindigkeit	Mittel (kann bei großen Projekten langsamer sein)	Schnell durch inkrementelle Builds	Langsam bei komplexen Builds
Flexibilität	Standardisiert, wenig flexibel	Sehr flexibel und anpassbar	Sehr anpassbar, aber komplex
IDE-Unterstützung	Standardisiert, wenig Flexibel	Gute Unterstützung in IntelliJ & Eclipse	Ja, aber weniger integriert

Ausgewähltes Tool: Gradle

Begründung:

- Bietet schnelle Build-Zeiten durch inkrementelle Builds
- Sehr gute Integration mit Spring Boot und flexibler anpassbar als klassische Tools

Prototyping der Benutzerschnittstelle

Kriterium	Moqups	Pencil	Figma
Plattform	Web-basiert	Desktop-Anwendung	Web-basiert

Kriterium	Moqups	Pencil	Figma
Interaktivität	Ja, klickbare Prototypen	Nein, statische Wireframes	Ja, klickbare Prototypen
Zusammenarbeit	Echtzeit-Kollaboration	Nur durch Datei-Export	Echtzeit-Kollaboration
Benutzerfreundlichkeit	Sehr einfach, Drag & Drop	Einfach, aber begrenzte Features	Intuitiv, viele Design-Features
Kosten	Kostenpflichtige Features	Kostenlos	Kostenlos (mit kostenpflichtigen Pro-Features)
Export-Möglichkeiten	PDF, PNG, HTML	PNG, SVG, PDF	PNG, SVG, PDF

Ausgewähltes Tool: Figma

Begründung:

- Kostenlos nutzbar und plattformunabhängig
- Teammitglieder haben bereits Erfahrung mit dem Tool

IDE / Editor

Kriterium	Eclipse	IntelliJ IDEA	NetBeans
Funktionalität	Umfangreich, viele Plugins	Sehr umfangreich, speziell für Java	Guter Funktionsumfang
Benutzerfreundlichkeit	Mittel, komplexe Oberfläche	Sehr intuitiv, modernes UI	Einfach, aber etwas veraltet
Erweiterbarkeit	Sehr hoch	Hoch	Mittel
Integration	Sehr gut mit Java-Umgebungen	Sehr gut mit modernen Tools	Gut, aber weniger verbreitet
Community & Support	Sehr gut mit Java-Umgebungen	Sehr gut mit modernen Tools	Mittelgroß
Kosten	Open Source, kostenlos	Kostenlose Community Edition, kostenpflichtige Ultimate Edition	Open Source, kostenlos

Ausgewähltes Tool: Eclipse

Begründung:

- Umfassende Unterstützung für Java, einschließlich Swing und Spring Boot
- Integration mit anderen ausgewählten Tools (z. B. Papyrus)
- Alle Teammitglieder sind mit diesem Tool bereits vertraut

Test-Automatisierung

Kriterium	JUnit	TestNG	AssertJ Swing
Funktionalität	Speziell für Unit-Tests	Unterstützt verschiedene Testarten	Automatisierte GUI-Tests für Swing
Benutzerfreundlichkeit	Einfach zu verwenden	Flexibel, aber komplexer als JUnit	Einfache API aber initiale Einrichtung erforderlich
Erweiterbarkeit	Modular, erweiterbar	Sehr flexibel, unterstützt verschiedene Frameworks	Unterstützt GUI-Komponenten-Tests mit erweiterten Features
Integration	Nahtlos in Java-Projekte integriert	Gute Integration mit Java und anderen Tools	Integriert sich gut mit JUnit für UI-Tests
Community & Support	Sehr groß	Groß, aber weniger verbreitet als JUnit	Kleinere Community
Kosten	Open Source, kostenlos	Open Source, kostenlos	Open Source, kostenlos

Ausgewähltes Tool: JUnit & AssertJ Swing

Begründung:

- JUnit perfekt für Unit-Tests, während AssertJ Swing ideal für automatisierte UI-Tests von Swing-Anwendungen ist
- Einfache Integration in bestehende Java-Projekte und Kombination möglich

Dokumentationstool

Kriterium	Javadoc	Doxygen	Sphinx
Sprache	Java	C, C++, Java, Python, viele weitere	Python, C/C++, andere mit Plugins
Integration	In Java-Compiler integriert	Eigenständig, IDE-Plugins	Python-basiert, mit vielen Erweiterungen
Ausgabeformate	HTML	HTML, LaTeX, RTF, man pages, XML	HTML, PDF, LaTeX, ePub
Kommentarsyntax	<code>/** ... */</code>	<code>/*! ... */</code> mit speziellen Tags	reStructuredText / Markdown
Popularität	Sehr verbreitet in Java-Projekten	Sehr verbreitet für C/C++	Standard in vielen Python-Projekten

Kriterium	Javadoc	Doxygen	Sphinx
Besonderheiten	Einfach in Java-Projekten zu verwenden	Sehr flexibel, viele Sprachen	Starke Erweiterbarkeit, schöne Ausgabe möglich
Erweiterbarkeit	Eingeschränkt	Hoch (Filter, Skripte)	Sehr hoch (Themes, Extensions)

Ausgewähltes Tool: Javadoc

Begründung:

- Als Standard im Java-Umfeld einfach in Java-Compiler integriert
- Unterstützt eine klare und umfassende Dokumentation direkt im Code

Obfuscator

Kriterium	Proguard	JavaGuard	yGuard
Sprache	Java, Android	Java	Java
Funktion	Obfuskation, Shrinking, Optimierung	Nur Obfuskation	Obfuskation, Shrinking
Integration	Android Build-Toolchain	Eigenständig	Plugin für Gradle, Maven, Ant
Popularität	Sehr verbreitet (Standard in Android)	Eher wenig verbreitet	Weit verbreitet im Java-Bereich
Konfiguration	proguard-rules.pro	XML-Dateien	XML-Dateien, einfache Konfiguration
Erweiterbarkeit	Durch z. B. R8 oder externe Tools	Eingeschränkt	Modular, erweiterbar
Lizenzmodell	Open Source	Open Source	Open Source

Ausgewähltes Tool: yGuard

Begründung:

- Speziell für Java-Anwendungen entwickelt
- Einfache Integration mit Gradle
- Aktiv gepflegt und mit modernen Java-Versionen kompatibel

Codeconventions

Kriterium	Oracle Java	Google Java	Mozilla Java
Einrückung	4 Leerzeichen	2 Leerzeichen	4 Leerzeichen
Max. Zeilenlänge	80 Zeichen	100 Zeichen	99 Zeichen

Kriterium	Oracle Java	Google Java	Mozilla Java
Veröffentlichung	1997, kaum aktualisiert	Aktuell gepflegt	Basierend auf Oracle, nicht mehr aktiv gepflegt
Namenskonvention	PascalCase, camelCase	PascalCase, camelCase	PascalCase, camelCase
Dokumentation	Javadoc verpflichtend	Javadoc empfohlen	Javadoc empfohlen
Besonderheiten	Rückwärtskompatibel, aber veraltet	Strenge Regeln für Lesbarkeit	Mozilla-spezifische Anpassungen

Ausgewählte Richtlinien: Google Java Style Guide

Begründung:

- Modern und aktuell gepflegt
- Gut lesbar und einheitlich
- Einfacher umzusetzen mit Tools wie Checkstyle

Kollaborationstool

Kriterium	Discord	Treffen im Praktikum	Email
Kommunikation	Sprach-, Video- und Text-Chat in Echtzeit	Persönliche Gespräche	Nur asynchrone Kommunikation
Flexibilität	Jederzeit erreichbar, auch mobil	Feste Zeiten, erfordert Anwesenheit	Nachrichten jederzeit versendbar, aber langsame Antworten
Dateiaustausch	Direkter Upload und Sharing	Vor Ort über USB/Sticks oder Ausdrücke (bzw. nach Absprache über anderes Tool)	Anhänge, aber unübersichtlich bei vielen Versionen
Zusammenarbeit	Kanäle und Threads für strukturierte Themen	Direkter Austausch, aber nicht dokumentiert	Unübersichtlich, da oft lange Email-Ketten entstehen

Ausgewähltes Tool: Discord

Begründung:

- Echtzeit-Kommunikation über Sprache, Video und Text ermöglicht schnelle Abstimmungen
- Flexibel und jederzeit verfügbar, auch für spontane Diskussionen
- Strukturierte Zusammenarbeit durch Kanäle, Threads und Dateiaustausch

Entwicklungsumgebung - Übersicht

Funktion	Tool
Versionierung	Git
UML-Tool	Papyrus
Build-Tool	Gradle
Prototyping	Figma
IDE / Editor	Eclipse
Testing	JUnit / AssertJ Swing
Dokumentation	Javadoc
Obfuscator	yGuard
Codeconventions	Google Java Style Guide
Kollaborationstool	Discord